Java Programming Assignment Answers

1. Explain Java's evolution from C/C++ and the role of bytecode, JVM and the Internet in Java's design.

Java's Evolution from C/C++:

Java was developed by James Gosling and his team at Sun Microsystems (released in 1995). It was designed with a syntax heavily derived from C and C++ to make it immediately familiar to programmers. However, Java discarded several complex, error-prone features of C and C++:

• No Explicit Pointers: Java eliminates pointer arithmetic, preventing direct memory access and reducing security vulnerabilities and memory corruption bugs.

• No Multiple Inheritance of Classes: To avoid the "Diamond Problem," Java restricts classes to single inheritance, utilizing interfaces to achieve multiple inheritance behavior.

• Automatic Memory Management: Unlike C++ where developers must manually manage memory via new and delete (risking memory leaks and dangling pointers), Java introduced automated garbage collection.

• Platform Independence: While C/C++ compile directly into platform-specific machine code, Java was engineered around the philosophy of "Write Once, Run Anywhere" (WORA).

Role of Bytecode, JVM, and the Internet:

• Bytecode: When a Java source file (.java) is compiled, the Java compiler (javac) does not generate machine-specific executable code. Instead, it generates an intermediate, portable format known as bytecode (.class files). Bytecode is standardized and machine-independent.

• JVM (Java Virtual Machine): The JVM is an abstract computing machine that acts as an execution engine. It interprets or compiles bytecode into native machine instructions for the underlying hardware platform (Windows, Linux, macOS, etc.). This makes Java applications completely platform-independent.

• The Internet: In the mid-1990s, the rise of the World Wide Web demanded secure, portable, and network-downloadable executable code (initially popularized through Java Applets). Java's compact bytecode format, secure sandbox execution model, and network-neutral design made it the ideal language for distributed, web-based applications.

2. Explain garbage collection, object reachability and the role of new in object creation.

Garbage Collection and Object Reachability:

• Garbage Collection (GC): In Java, garbage collection is the automated process of reclaiming dynamic memory by destroying objects that are no longer needed or reachable by the running program. This prevents memory leaks and frees developers from manual memory deallocation.

• Object Reachability: An object is considered "reachable" if it can be accessed by any active live thread in the program, either directly through a reference variable or indirectly through a chain of references from other reachable objects. Once an object has no active references pointing to it, it becomes "unreachable" (eligible for garbage collection). The JVM periodically runs its garbage collector to sweep away these unreachable objects and free up heap memory.

Role of new in Object Creation:

The new operator is a keyword used in Java to dynamically allocate memory for a new object at runtime. When executed:

1. Memory Allocation: The JVM allocates sufficient memory on the heap to store the object's instance variables.
2. Default Initialization: All instance variables are initialized to their default values (e.g., 0 for numbers, false for booleans, null for object references).
3. Constructor Invocation: The new operator immediately invokes the appropriate class constructor to initialize the newly created object with user-defined values.
4. Reference Return: Finally, new returns the memory address (reference) of the newly created object, which is typically assigned to a reference variable.

3. Trace a Java expression containing arithmetic, relational, logical, assignment and increment operators, showing operator precedence.

Consider the following Java expression:

```
int a = 5, b = 10, c = 15;
boolean result = ++a + b > c && (b++ < c || a == c);
```

Step-by-Step Trace & Operator Precedence:

1. Pre-increment (++a):
   - Operator: Unary increment (++)
   - Precedence: Highest among operators in the expression.
   - Action: a is incremented from 5 to 6. The expression evaluates ++a as 6.
   - Current state: a = 6, b = 10, c = 15.

2. Arithmetic Addition (++a + b):
   - Operator: Arithmetic addition (+)
   - Precedence: Higher than relational operators.
   - Action: 6 + 10 = 16.

3. Relational Greater-Than (16 > c):
   - Operator: Relational greater-than (>)
   - Precedence: Higher than logical AND (&&).
   - Action: 16 > 15 evaluates to true.
   - Current state: true && (b++ < c || a == c)

4. Right Sub-expression Evaluation - Parentheses first ((b++ < c || a == c)):

• Parentheses () have the highest grouping precedence.
• Inside parentheses, check left-to-right:
a. Post-increment comparison (b++ < c):
- Evaluates b (10) < c (15) -> true.
- Side effect: b is incremented to 11 (b = 11).
b. Short-circuit evaluation: Because the left side of the logical OR (||) is true, the right side (a == c) is short-circuited and not evaluated.
c. Result of parentheses: true.

5. Logical AND (true && true):
• Operator: Logical AND (&&)
• Action: true && true evaluates to true.

6. Assignment (=):
• Operator: Assignment (=)
• Precedence: Lowest precedence.
• Action: result = true.

Final Output: result = true, a = 6, b = 11, c = 15.

4. Design a Student class using fields, constructors, methods, access modifiers and this keyword.

Java Implementation:

```
public class Student {
    // Private fields (Encapsulation)
    private String name;
    private int id;
    private double gpa;

    // Default Constructor
    public Student() {
        this("Unknown", 0, 0.0); // Constructor chaining using this()
    }

    // Parameterized Constructor
    public Student(String name, int id, double gpa) {
        this.name = name; // 'this' resolves shadowing between field and parameter
        this.id = id;
        this.gpa = gpa;
    }

    // Getter and Setter methods
    public String getName() {
        return name;
    }
```

```
    public void setName(String name) {
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public double getGpa() {
        return gpa;
    }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        }
    }

    // Instance Method
    public void displayStudentInfo() {
        System.out.println("Student ID: " + this.id + ", Name: " + this.name + ", GPA: "
+ this.gpa);
    }
}
```

5. Develop a small Java application combining constructors, overloaded methods, recursion and encapsulated data.

Java Implementation:

```
class MathOperations {
    // Encapsulated field
    private int baseNumber;

    // Constructor
    public MathOperations(int baseNumber) {
        this.baseNumber = baseNumber;
    }

    public int getBaseNumber() {
        return baseNumber;
    }

    // Overloaded Method 1: Computes factorial using baseNumber
    public long computeFactorial() {
        return calculateFactorialRecursive(this.baseNumber);
```

```
    }

    // Overloaded Method 2: Computes factorial for any given integer n using
recursion
    public long computeFactorial(int n) {
        return calculateFactorialRecursive(n);
    }

    // Recursive helper method
    private long calculateFactorialRecursive(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("Number must be non-negative.");
        }
        if (n == 0 || n == 1) {
            return 1;
        }
        return n * calculateFactorialRecursive(n - 1);
    }
}

public class MainApp {
    public static void main(String[] args) {
        // Instantiate object with encapsulated data
        MathOperations math = new MathOperations(5);

        // Using method overloading (No-arg version uses encapsulated field)
        long result1 = math.computeFactorial();
        System.out.println("Factorial of encapsulated base (" +
math.getBaseNumber() + ") = " + result1);

        // Using overloaded recursive method with parameter
        long result2 = math.computeFactorial(7);
        System.out.println("Factorial of 7 using overloaded method = " + result2);
    }
}
```